```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras import models, layers, optimizers, datasets, backend

from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
```

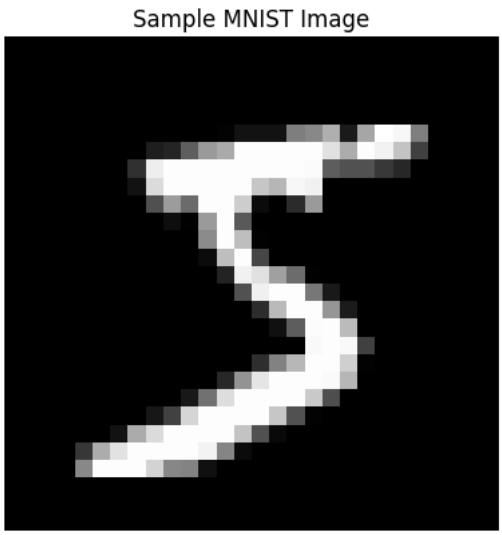
```
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

```
plt.imshow(x_train[0], cmap='gray')
plt.title("Sample MNIST Image")
plt.axis('off')
plt.show()

print("Label:", y_train[0])
```




```
Label: 5
```

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
x_train = x_train.reshape(x_train.shape[0], 784)
x_test = x_test.reshape(x_test.shape[0], 784)

print("Training shape after flattening:", x_train.shape)
print("Testing shape after flattening:", x_test.shape)
```

```
Training shape after flattening: (60000, 784)
Testing shape after flattening: (10000, 784)
```

```
lb = LabelBinarizer()

y_train = lb.fit_transform(y_train)
y_test = lb.transform(y_test)

print("Label shape:", y_train.shape)
```

```
Label shape: (60000, 10)
```

```
model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation='sigmoid'),
    layers.Dense(64, activation='sigmoid'),
    layers.Dense(10, activation='softmax')
])
```

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	100,480
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)

```
optimizer = optimizers.SGD(learning_rate=0.01)

model.compile(
    optimizer=optimizer,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	100,480
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)

```
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)
```

```
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 5s 8ms/step - accuracy: 0.1830 - loss: 2.2844 - val_accuracy: 0.2727 - val_loss: 2.2501
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.3206 - loss: 2.2250 - val_accuracy: 0.3113 - val_loss: 2.1952
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.4132 - loss: 2.1636 - val_accuracy: 0.4278 - val_loss: 2.1220
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.4820 - loss: 2.0803 - val_accuracy: 0.4967 - val_loss: 2.0215
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.5235 - loss: 1.9678 - val_accuracy: 0.5830 - val_loss: 1.8886
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.5722 - loss: 1.8251 - val_accuracy: 0.6170 - val_loss: 1.7282
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6086 - loss: 1.6621 - val_accuracy: 0.6782 - val_loss: 1.5532
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6537 - loss: 1.4956 - val_accuracy: 0.7025 - val_loss: 1.3832
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6891 - loss: 1.3413 - val_accuracy: 0.7468 - val_loss: 1.2318
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.7227 - loss: 1.2080 - val_accuracy: 0.7705 - val_loss: 1.1033
```

Double-click (or enter) to edit

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7451 - loss: 1.1277
Test Loss: 1.1277387142181396
Test Accuracy: 0.7451000213623047
```

```
predictions = model.predict(x_test)

predicted_classes = np.argmax(predictions, axis=1)
actual_classes = np.argmax(y_test, axis=1)

print("Predicted:", predicted_classes[:10])
print("Actual:   ", actual_classes[:10])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step
Predicted: [7 2 1 0 4 1 7 4 6 7]
Actual:    [7 2 1 0 4 1 4 9 5 9]
```

```
print(classification_report(
    actual_classes,
    predicted_classes
))
```

```
              precision    recall  f1-score   support

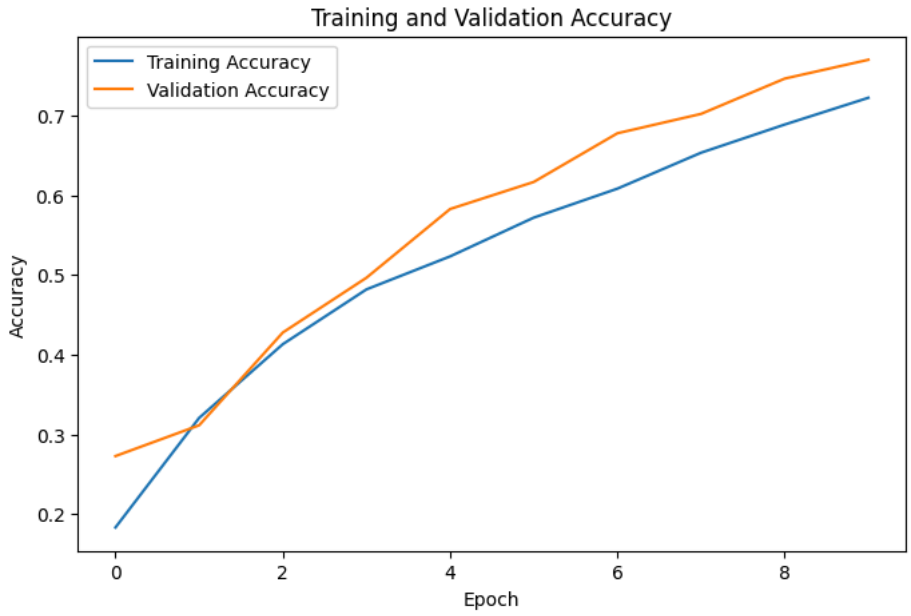
           0       0.77      0.97      0.86       980
           1       0.75      0.99      0.85      1135
           2       0.82      0.73      0.77      1032
           3       0.65      0.88      0.74      1010
           4       0.74      0.73      0.73       982
           5       0.87      0.37      0.52       892
           6       0.84      0.85      0.85       958
           7       0.64      0.89      0.75      1028
           8       0.86      0.49      0.62       974
           9       0.71      0.48      0.57      1009

    accuracy                           0.75     10000
   macro avg       0.76      0.74      0.73     10000
weighted avg       0.76      0.75      0.73     10000
```

```
plt.figure(figsize=(8, 5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

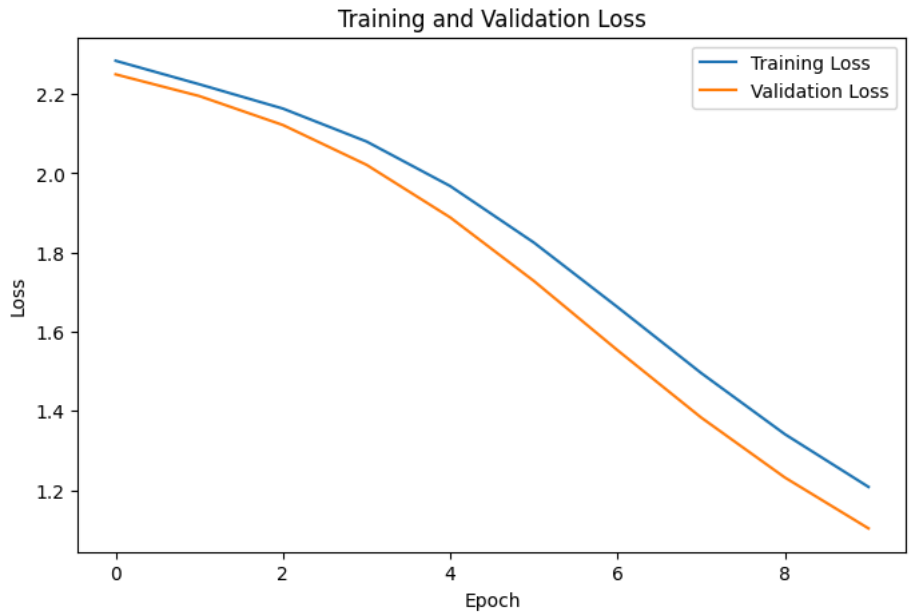



```
plt.figure(figsize=(8, 5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Training and Validation Loss')
```

```
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



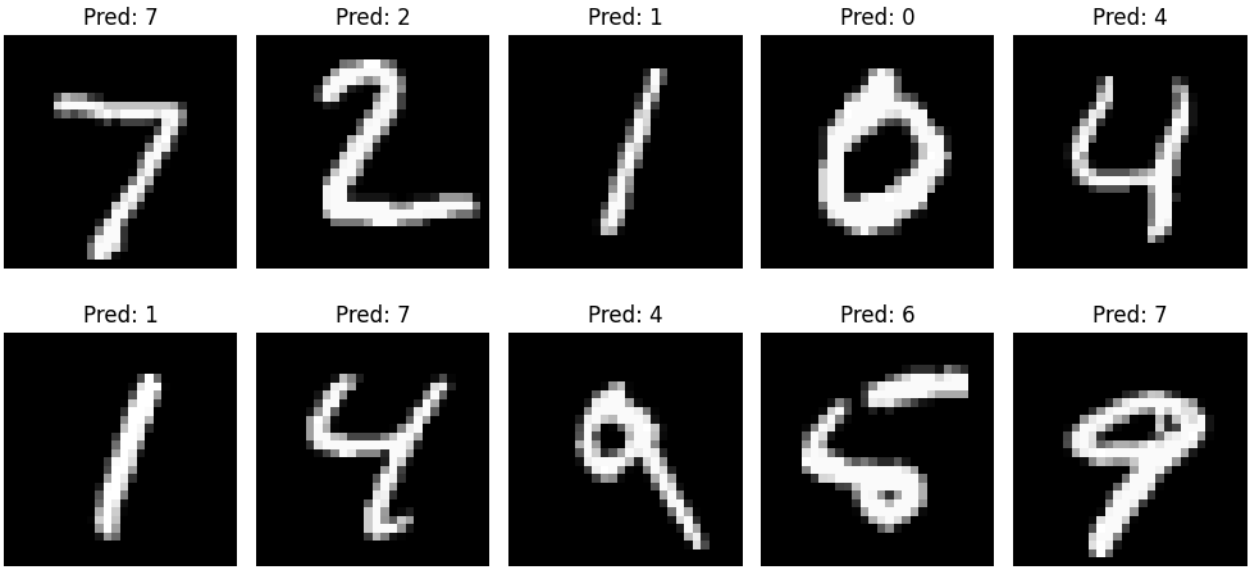

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)

    image = x_test[i].reshape(28, 28)
    plt.imshow(image, cmap='gray')

    plt.title(f"Pred: {predicted_classes[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```



Start coding or generate with AI.